Guía De Laboratorio 10

Sistema Inteligente de Apoyo Pedagógico y Gestión del Aprendizaje para Instituciones Educativas

INTEGRANTES – GRUPO 6:

Rivera Castañeda, Jesús Martín	- U21213646
Quispe Fernández, Alonso	- U20208103
Alanya Guevara, Ben Andre	- U21315933
Ninatanta Fernández, Renato Ventura	- U21205210

ASIGNATURA:

Curso Integrador II: Software

DOCENTE:

Barturen Huaman, Renzo Renato

SECCIÓN:

31738

Lima – Perú

Índice

1. Resultado y alcance del incremento 3
1.1 Producto mínimo de la semana 3
2. Correspondencia con el sílabo 3
3. Fundamentos de calidad funcional 3
3.1 Adecuación funcional en ISO/IEC 25010 3
3.2 Verificación y validación 4
3.3 Requisito, criterio, oráculo y evidencia 4
3.4 Capas de prueba y herramientas 4
3.5 Métricas operativas del laboratorio 5
3.6 Criterios de salida 5
3.7 Uso controlado de inteligencia artificial 6
4. Arquitectura del incremento 6
5. Sesión 19: evaluación de la adecuación funcional 7
5.1 Actividad 1 — Establecer la línea base funcional 7
5.2 Actividad 2 — Crear la matriz funcional ejecutable 7
5.3 Actividad 3 — Fortalecer las pruebas de integración 8
5.4 Actividad 4 — Medir la eficacia con pruebas por mutación 8
5.5 Actividad 5 — Interpretar la calidad sin inflar la conclusión 9
6. Sesión 20: verificación, validación e IA supervisada 9
6.1 Actividad 6 — Verificar el contrato OpenAPI 9
6.2 Actividad 7 — Validar el objetivo de usuario con Playwright 9
6.3 Actividad 8 — Generar casos con IA y revisar cada propuesta 10
6.4 Actividad 9 — Ejecutar el gate funcional 10
6.5 Actividad 10 — Emitir el informe de calidad funcional 10
7. Informe de calidad funcional 10
8. Riesgos conectados al incremento 11
9. Entregables y estado de evidencia (E0–E11) 11
10. Checklist de cierre 12

1. Resultado y alcance del incremento

Este avance corresponde a la Semana 10 del Curso Integrador II: Software, con la que inicia la Unidad 3 (calidad de software orientada a la funcionalidad). Toma la versión 1 de ESTUD-IA desplegada al cierre de la Semana 9 y comprueba, con resultados reproducibles, si cubre el alcance funcional acordado, produce respuestas correctas y permite completar el objetivo principal sin atajos. La referencia conceptual es ISO/IEC 25010:2023.

Se trabaja sobre la misma aplicación React + Laravel + MySQL. La ejecución combina una matriz de trazabilidad, pruebas de integración con MySQL real, pruebas por mutación, comprobación del contrato OpenAPI, pruebas de navegador y un gate automatizado. La IA ayuda a proponer escenarios y borradores de prueba, pero cada resultado se revisa, se ejecuta y se contrasta con un oráculo definido por personas.

1.1 Producto mínimo de la semana

- Matriz funcional ejecutable (JSON) con validador que detecta requisitos sin ID, criterio, subcaracterística o prueba.
- Pruebas de integración FunctionalSuitability sobre MySQL real, casos API-F01 a API-F05.
- Reporte de pruebas por mutación (Infection) acotado a la lógica de envío de intentos, con un mutante sobreviviente analizado.
- Contrato OpenAPI del perfil de calidad y ejecución de Schemathesis solo sobre GET públicos.
- Pruebas Playwright WEB-F01 y WEB-F02 sobre React y Laravel reales, sin interceptar la red ni usar skip.
- Brief de IA, plan, decisiones de aceptación o rechazo y diff revisado.
- Gate local y workflow de CI con reportes como artefactos, más el informe firmado por el equipo.

Alcance deliberadamente acotado: esta guía no adelanta las métricas de compatibilidad, accesibilidad o facilidad de aprendizaje (Semanas 11 y 12) ni la latencia y capacidad (Unidad 4).

2. Correspondencia con el sílabo

Sesión	Tema del sílabo	Aplicación en ESTUD-IA
19	Calidad de software orientada a la funcionalidad según ISO 25010. Funcionalidad completa, correcta e idónea.	Definir el alcance funcional medible, trazar requisitos con pruebas y evaluar resultados y mutaciones.
20	Calidad orientada a la funcionalidad. Verificación y validación. Pruebas de funcionalidad.	Comprobar contrato y API, validar el flujo crítico en navegador y automatizar el gate con IA supervisada.

3. Fundamentos de calidad funcional

3.1 Adecuación funcional en ISO/IEC 25010

El estándar define el modelo; no impone las fórmulas ni los umbrales de este laboratorio. La adecuación funcional examina si las funciones cubren las tareas especificadas y si entregan resultados correctos y útiles para el uso previsto.

Subcaracterística	Pregunta aplicada a ESTUD-IA	Evidencia suficiente
Completitud funcional	¿Están cubiertas todas las funciones del alcance: catálogo, contenidos, autenticación y envío de intento con prevención de duplicados?	Cada requisito se vincula con al menos una prueba ejecutada y no queda una función crítica sin implementar.
Corrección funcional	¿La aplicación devuelve el estado, los datos y los efectos persistentes esperados para entradas válidas e inválidas?	Aserciones sobre HTTP, JSON, filas, auditoría y concurrencia coinciden con reglas explícitas.
Idoneidad funcional	¿Un Alumno puede completar el objetivo de enviar un intento sin llamadas manuales, datos ocultos ni pasos ajenos al flujo?	El recorrido de navegador empieza en la búsqueda, termina en la confirmación y no requiere workaround.

La idoneidad funcional no equivale a usabilidad ni a rendimiento: esas dimensiones se evalúan en semanas posteriores.

3.2 Verificación y validación

Actividad	Pregunta	Ejemplo en ESTUD-IA
Verificación	¿El producto cumple una condición especificada?	POST de intento devuelve 201 para Alumno, 403 para Docente y 409 cuando el ejercicio ya tiene un intento.
Validación	¿El producto permite cumplir la necesidad de uso prevista?	El Alumno busca contenidos, se identifica, envía su intento y recibe una confirmación visible usando solo la interfaz.

3.3 Requisito, criterio, oráculo y evidencia

Un requisito declara el comportamiento esperado; el criterio lo vuelve observable; el oráculo permite decidir si el resultado coincide; la evidencia conserva la ejecución. Si falta uno de los cuatro, el estado Aprobado no puede justificarse.

Elemento	Ejemplo
Requisito	RF-INT-02: la aplicación impide registrar dos intentos activos del mismo alumno para el mismo ejercicio.
Criterio	Dos solicitudes concurrentes producen una respuesta 201, una 409 y una sola fila en intento_ejercicio.
Oráculo	Códigos HTTP exactos y SELECT COUNT(*) igual a 1.
Prueba	API-F05 (solicitudes concurrentes no duplican el intento).
Evidencia	Reporte de PHPUnit, salida del caso y consulta sobre la base de pruebas.

3.4 Capas de prueba y herramientas

Capa	Herramienta	Qué demuestra	Qué no demuestra
Unidad	JUnit/AssertJ → PHPUnit	Reglas de servicio aisladas y casos límite rápidos.	Integración HTTP, seguridad o esquema real.
Integración	Spring Boot + Testcontainers → Feature tests de Laravel con MySQL real	Controllers, Sanctum, transacciones y migraciones juntos.	Que el usuario complete el flujo en el navegador.
Contrato	springdoc + Schemathesis → Scramble (OpenAPI) + Schemathesis	Respuestas documentadas, esquema y rechazo de entradas generadas.	Semántica completa del negocio.
Extremo a extremo	Playwright → Playwright	Flujo crítico sobre React y API reales.	Cobertura exhaustiva de combinaciones.
Eficacia de pruebas	PIT → Infection	Si las pruebas detectan cambios artificiales en la lógica.	Ausencia de defectos.

3.5 Métricas operativas del laboratorio

Son una operacionalización didáctica del proyecto, no fórmulas normativas de ISO/IEC 25010.

Métrica	Cálculo	Lectura
Completitud	Requisitos del alcance con prueba aprobada ÷ requisitos del alcance × 100	Un requisito sin ejecución permanece no cubierto, aunque exista código.
Corrección	Requisitos evaluados cuyos oráculos críticos pasan ÷ requisitos evaluados × 100	Se calcula por requisito para no premiar muchas aserciones triviales.
Idoneidad	Objetivos de usuario completados sin workaround ÷ objetivos evaluados × 100	El objetivo crítico de envío de intento debe aprobar por completo.
Mutación	Mutantes detectados ÷ (detectados + sobrevivientes) × 100	Un sobreviviente puede revelar una aserción débil o ser equivalente.
Conformidad de contrato	Operaciones sin fallos de esquema o estado ÷ operaciones ejercitadas × 100	Solo informa sobre endpoints y entradas realmente ejecutadas.

3.6 Criterios de salida

Condición	Decisión
Un requisito crítico no tiene prueba ejecutada o aparece como skip	Bloquea el informe de calidad funcional.
Falla un oráculo de envío de intento, autorización, persistencia o concurrencia	Bloquea la versión hasta corregir y repetir la regresión.
Schemathesis encuentra 5xx o una respuesta incompatible con OpenAPI	Bloquea el contrato afectado; conservar el caso mínimo reproducible.

Condición	Decisión
Infection queda por debajo de 60 % en el paquete objetivo	Requiere revisar mutantes; el umbral es didáctico y no se reduce para aprobar.
La IA genera una prueba que cambia la expectativa, usa skip u oculta un fallo	Se rechaza la propuesta y se conserva la razón.
Una herramienta no puede ejecutarse	Marcar Bloqueado, conservar el diagnóstico y no atribuir un resultado.

3.7 Uso controlado de inteligencia artificial

La IA es útil para expandir escenarios y redactar un primer borrador de pruebas, pero no conoce el contrato verdadero ni decide qué comportamiento es correcto. Su salida se trata como un cambio de código no confiable hasta que una persona revisa el plan, el diff y la ejecución.

Riesgo	Control obligatorio
Endpoint, selector o librería inventados	Contrastar con OpenAPI, DOM accesible, composer.lock/package-lock y documentación oficial.
Prueba que pasa sin observar el resultado	Exigir una aserción específica sobre UI, HTTP, JSON o estado persistente.
Reparación que oculta el defecto	Rechazar skip, force, esperas arbitrarias y expectativas más laxas; revisar el diff antes de ejecutar.
Exposición de datos o secretos	Usar solo fixtures sintéticos; no enviar tokens de Sanctum, APP_KEY, .env ni logs al modelo.
Cambio no reproducible	Guardar el brief y el plan; versionar únicamente el código revisado y ejecutado.

4. Arquitectura del incremento

La guía oficial usa Spring Boot, Failsafe, PIT y springdoc. En ESTUD-IA se conserva la intención de cada pieza con las herramientas reales del stack.

Archivo de la guía	Equivalente en ESTUD-IA	Responsabilidad
quality/week10/requirements-functional.json	Igual (agnóstico al framework)	Declarar alcance, criterios observables y pruebas asociadas.
scripts/validate-quality-matrix.mjs	Igual	Detectar requisitos sin ID, criterio, subcaracterística o prueba.

Archivo de la guía	Equivalente en ESTUD-IA	Responsabilidad
FunctionalSuitabilityIT.java	backend/tests/Feature/Integration/FunctionalSuitabilityTest.php	Probar el flujo con Laravel real y MySQL de pruebas.
pom.xml con PIT y springdoc	backend/composer.json con infection/infection y dedoc/scramble; infection.json5	Mutación acotada y exportación del contrato OpenAPI.
frontend/e2e/functional-suitability.spec.ts	Igual, apuntando a rutas y labels de React	Validar el objetivo de usuario con API y frontend reales.
quality/week10/ai-test-brief.md	Igual	Limitar y auditar la generación asistida de casos.
scripts/week10-quality-gate.sh y workflow CI	Mismo patrón con composer, php artisan test, infection y un servicio MySQL en el runner	Ejecutar y repetir las comprobaciones en un entorno limpio.

5. Sesión 19: evaluación de la adecuación funcional

Entorno local controlado. Al finalizar deben existir una matriz válida, pruebas de integración ejecutadas, un reporte de mutación acotado y una interpretación basada en requisitos, no en cantidad de casos.

5.1 Actividad 1 — Establecer la línea base funcional

- Registrar el commit de la versión 1 (apf2-<hash>) sin borrar cambios desconocidos.
- Ejecutar el gate de la Semana 9 o, como mínimo, php artisan test, npm test, npm run build y el smoke local.
- Crear quality/week10 y evidence/week10, y verificar que PHP, Composer, Node, Docker y Playwright corresponden a los lockfiles.

Si una prueba antigua falla, no se elimina ni se atribuye a la Semana 10 sin reproducir la causa. Evidencia E1: commit, versiones, salida del gate previo y lista de bloqueos iniciales.

5.2 Actividad 2 — Crear la matriz funcional ejecutable

La matriz contiene solo comportamiento comprometido para la versión evaluada; no se añaden funciones futuras para aumentar el alcance. Alcance: buscar contenidos sintéticos, autenticarse y enviar el intento de un ejercicio.

ID	Subcaracterística	Crítico	Criterio observable	Pruebas
RF-CAT-01	completeness	Sí	GET /api/v1/cursos responde 200 y contiene MAT101 y COM101.	API-F01, WEB-F01
RF-CON-01	completeness	Sí	MAT101 para 2026-09-03 responde 200 y ofrece los dos contenidos del fixture.	API-F01, WEB-F01
RF-AUT-01	correctness	Sí	Credenciales válidas producen un token de Sanctum y credenciales inválidas producen 401 genérico.	API-F02
RF-INT-01	correctness	Sí	Un Alumno envía un intento sobre un ejercicio libre con 201, código público del intento y evento de auditoría.	API-F02, WEB-F02
RF-INT-02	correctness	Sí	Anónimo (401), Docente (403), entrada inválida (422) y doble envío (409) se rechazan sin duplicar filas.	API-F03, API-F04, API-F05
RF-USO-01	appropriateness	Sí	El Alumno busca, elige, se identifica, envía y ve la confirmación sin llamadas manuales.	WEB-F02

El validador termina con código cero y muestra MATRIX OK. Solo valida estructura y trazabilidad declarada; todavía no demuestra que las pruebas pasen. Evidencia E2: matriz JSON y salida del validador.

5.3 Actividad 3 — Fortalecer las pruebas de integración

FunctionalSuitabilityTest.php usa RefreshDatabase sobre una base MySQL de pruebas dedicada, de modo que cada caso comienza con un estado conocido y no toca la base de desarrollo ni la desplegada. Los usuarios (Alumno y Docente) se crean con Hash::make y los tokens se emiten con createToken de Sanctum, excepto la prueba de login, que usa el endpoint real.

Caso	Oráculo principal	Aporte
API-F01	Catálogo y contenidos exactos del fixture.	Completitud y corrección de lectura.
API-F02	Login 200, intento 201, código público y evento de auditoría.	Corrección del camino exitoso.
API-F03	401, 403 y 422 sin escritura.	Corrección de alternativas y validación.
API-F04	201 seguido de 409 y una fila.	Regla de unicidad observable.
API-F05	201 y 409 concurrentes con una fila (lockForUpdate + UNIQUE).	Corrección bajo solicitudes simultáneas.

5.4 Actividad 4 — Medir la eficacia con pruebas por mutación

Infection modifica el código (invierte una condición, reemplaza un valor de retorno) y ejecuta las pruebas; si ninguna falla, el mutante sobrevive. Un sobreviviente no es automáticamente un defecto: puede ser una mutación equivalente. Se acota primero a app/Services/IntentoEjercicioService.php, con piso de MSI de 60 % solo en ese paquete. Si no existen pruebas unitarias del servicio, se crean primero y Infection se marca Bloqueado hasta que encuentre clases y casos válidos. Se analiza un mutante sobreviviente útil, se añade una aserción que represente una regla real y se repite la ejecución.

Evidencia E4: reporte inicial, mutante analizado, prueba fortalecida y reporte repetido.

5.5 Actividad 5 — Interpretar la calidad sin inflar la conclusión

- Contar requisitos del alcance y los que tienen al menos una prueba aprobada; calcular la completitud operativa.
- Calcular la corrección a nivel de requisito, no por cantidad de aserciones.
- Registrar el puntaje de mutación y clasificar cada sobreviviente como prueba débil, código no cubierto o mutante equivalente probable.
- Redactar cinco a ocho líneas con versión, alcance, resultados y límites.

Afirmación incorrecta	Redacción verificable
El sistema tiene 80 % de calidad.	Cinco de seis requisitos del alcance tienen prueba aprobada; RF-USO-01 sigue pendiente de validación en navegador.
La cobertura demuestra que no hay errores.	La cobertura confirma ejecución de líneas; las pruebas de integración e Infection aportan evidencia sobre oráculos y eficacia.
La IA comprobó todos los casos.	La IA propuso tres escenarios; dos se aceptaron tras revisión y ejecución, y uno se rechazó por inventar un endpoint.

Evidencia E5: cálculos, clasificación de mutantes y conclusión con alcance explícito.

6. Sesión 20: verificación, validación e IA supervisada

La segunda sesión contrasta la API con su contrato, valida el objetivo crítico desde el navegador y automatiza el conjunto, siempre en entorno local o de pruebas con datos sintéticos.

6.1 Actividad 6 — Verificar el contrato OpenAPI

- Generar el contrato con Scramble (dedoc/scramble) y habilitarlo solo en el perfil de calidad mediante una variable de entorno; la documentación interna no se expone en producción por defecto.
- Confirmar que el documento contiene /api/v1/cursos y /api/v1/contenidos y exportarlo a evidence/week10/openapi.json.
- Ejecutar Schemathesis con versión exacta solo sobre esos GET públicos y no destructivos, con un máximo de ejemplos acotado. Nunca dirigir generación de entradas contra el POST de intentos ni contra producción.

Un fallo puede estar en la implementación o en OpenAPI; no se corrige el contrato para describir un defecto si la regla de negocio exige otro resultado. Evidencia E6: contrato exportado, versión de Schemathesis y salida completa.

6.2 Actividad 7 — Validar el objetivo de usuario con Playwright

- WEB-F01 compara la cantidad de contenidos recibida de la API con los elementos mostrados en React.
- WEB-F02 recorre buscar, elegir, ingresar, enviar el intento y ver la confirmación, una sola vez sobre un ejercicio sintético libre. No usa test.skip: si falta el ejercicio o la contraseña, falla con un diagnóstico explícito.

- La contraseña sintética del Alumno se introduce por variable de entorno, nunca en el archivo ni en el historial del shell. Si el fixture ya fue consumido, se prepara otro entorno sintético o se registra Bloqueado; no se borran datos de producción.

Evidencia E7: reporte Playwright con WEB-F01 y WEB-F02 ejecutados y sin casos omitidos.

6.3 Actividad 8 — Generar casos con IA y revisar cada propuesta

El brief contiene solo requisitos, datos sintéticos, selectores accesibles y acciones permitidas. El planner propone un plan para RF-CAT-01, RF-CON-01 y RF-USO-01 sin escribir código; el equipo revisa cada escenario contra la matriz; el generator implementa solo los escenarios marcados como ACEPTADOS; el healer se usa únicamente para diagnosticar.

Control de revisión	Aceptar cuando	Rechazar cuando
Trazabilidad	El título cita un requisito y el oráculo es observable.	El caso solo navega o toma una captura.
Fidelidad	Usa rutas, campos y estados presentes en OpenAPI y en la UI de React.	Inventa endpoint, selector, rol o dato.
Estabilidad	Usa roles, labels, autoespera y datos controlados.	Agrega sleep, selector frágil o dependencia del orden.
Integridad	Mantiene la expectativa y falla ante el defecto.	Usa skip, force o cambia el estado esperado para pasar.
Privacidad	Solo recibe fixtures y credenciales ficticias omitidas.	Incluye token, .env, dato real o log sin depurar.

La adopción queda registrada con tres elementos: plan original, decisión Aceptado o Rechazado con motivo, y diff final ejecutado. El modelo, por sí mismo, no produce evidencia de calidad. Evidencia E8: brief, plan, decisiones, diff y reporte de las pruebas aceptadas.

6.4 Actividad 9 — Ejecutar el gate funcional

Una sola secuencia que se detiene ante la primera condición no satisfecha: validar la matriz, ejecutar PHPUnit (unitarias e integración), Infection sobre el paquete objetivo, el contrato con Schemathesis, el build y las pruebas de React, y Playwright. Se repite en un runner limpio con un workflow de GitHub Actions de permisos mínimos y un servicio MySQL, publicando los reportes como artefactos.

Un gate verde significa que las condiciones codificadas pasaron en esa ejecución; no que el producto carezca de defectos ni que las funciones fuera del alcance hayan sido evaluadas. Si el gate pasa local y falla en CI, se comparan versiones, variables y archivos no versionados. Evidencia E9: salida del gate local, ejecución CI y reportes.

6.5 Actividad 10 — Emitir el informe de calidad funcional

La tabla se completa con valores obtenidos, no estimados. Un enlace de evidencia vacío mantiene el requisito Pendiente.

7. Informe de calidad funcional

Indicador	Resultado	Evidencia	Interpretación
Completitud funcional	___ de ___ = ___ %	Matriz y reportes	Funciones cubiertas dentro del alcance.
Corrección funcional	___ de ___ = ___ %	PHPUnit y Schemathesis	Requisitos cuyos oráculos críticos pasan.
Idoneidad funcional	___ de ___ objetivos	Playwright WEB-F02	Objetivo completado sin workaround.
Puntaje de mutación	___ %	Reporte de Infection	Eficacia de pruebas del paquete objetivo.
Casos de IA	___ aceptados y ___ rechazados	Plan, decisiones y diff	Aporte revisado; no es una métrica del producto.

Los defectos y bloqueos se describen por síntoma, condición, resultado real, resultado esperado y evidencia. El informe cierra con el alcance no evaluado y la decisión técnica del equipo: Aprobada, Condicionada o Bloqueada. Todos los valores quedan en blanco hasta ejecutar las pruebas.

8. Riesgos conectados al incremento

Se retoman los riesgos R-01 a R-18 de los avances anteriores y se agregan dos propios de esta semana.

ID	Riesgo formulado	Disparador	Control / respuesta	Indicador
R-19	Si el contrato OpenAPI generado por Scramble se desincroniza de las rutas reales, Schemathesis podría aprobar o rechazar casos por una documentación incorrecta y no por un defecto del producto.	Schemathesis reporta estados no documentados que no se reproducen con una llamada manual.	Regenerar el contrato en cada ejecución del gate, reproducir cada fallo con un caso mínimo y decidir desde la regla de negocio qué lado corregir.	NF01
R-20	Si Infection se ejecuta sobre todo el proyecto o sin pruebas unitarias del servicio, podría tardar demasiado o dar un puntaje que no representa la lógica de envío de intentos.	La ejecución supera el tiempo del laboratorio o informa que no hay mutantes.	Acotar el paquete objetivo a IntentoEjercicioService, crear primero pruebas de la regla y marcar Bloqueado si no hay casos válidos.	DEL-01

9. Entregables y estado de evidencia (E0–E11)

ID	Entregable	Contenido mínimo	Estado
E0	Inicio controlado	Issue, rama feature/semana10-calidad-funcional y commit base.	Pendiente
E1	Línea base	Commit, versiones, salida del gate previo y bloqueos iniciales.	Pendiente
E2	Matriz funcional	JSON de requisitos y salida MATRIX OK del validador.	Pendiente

ID	Entregable	Contenido mínimo	Estado
E3	Integración	Reporte de PHPUnit con API-F01 a API-F05.	Pendiente
E4	Mutación	Reporte inicial, mutante analizado y reporte repetido.	Pendiente
E5	Interpretación	Cálculos, clasificación de mutantes y conclusión acotada.	Pendiente
E6	Contrato	openapi.json, versión de Schemathesis y salida completa.	Pendiente
E7	Validación en navegador	Reporte Playwright con WEB-F01 y WEB-F02 sin omisiones.	Pendiente
E8	IA supervisada	Brief, plan, decisiones Aceptado/Rechazado, diff y reporte.	Pendiente
E9	Gate	Salida local, ejecución CI y artefactos.	Pendiente
E10	Informe	Informe firmado con versión, alcance, métricas, defectos, límites y decisión.	Pendiente
E11	Defensa	Presentación y Pull Request con revisión por pares.	Pendiente

10. Checklist de cierre

- La matriz contiene únicamente funciones comprometidas para la versión evaluada.
- Cada requisito crítico tiene criterio, oráculo, prueba ejecutada y evidencia enlazada.
- PHPUnit ejecutó API-F01 a API-F05 sobre MySQL real de pruebas, sin omisiones.
- Infection analizó el paquete correcto y los mutantes sobrevivientes fueron revisados.
- Schemathesis solo atacó endpoints y entorno autorizados y conservó el caso reproducible.
- Playwright ejecutó WEB-F01 y WEB-F02 sin skip ni respuestas simuladas.
- La IA no recibió secretos ni datos personales y ninguna reparación relajó el oráculo.
- El gate local y el workflow usan dependencias fijadas (composer.lock, package-lock.json) y reportes recuperables.
- El informe distingue resultados, límites y bloqueos; no afirma ausencia de defectos.
- Los riesgos R-19 y R-20 quedan vinculados al registro de riesgos de ESTUD-IA.